ALZEN V2.0 — ENTERPRISE AUTONOMOUS AGENCY

21-Phase Production Deployment Blueprint

> **Philosophy:** Don't reinvent the wheel. Wire Paperclip (The Boss) + Hermes (The Workforce) + Graphify (The X-Ray) into a single, self-improving, zero-human enterprise that runs, learns, and scales autonomously.

ARCHITECTURE OVERVIEW

```
┌──────────────────────────────────────────────────────────────────┐
│                        ALZEN V2.0 LAYERS                         │
│                                                                  │
│  LAYER 0 ── GitHub Repos + Docker Infrastructure + OS Baseline   │
│  LAYER 1 ── Graphify Knowledge Graph (Global Codebase X-Ray)     │
│  LAYER 2 ── Paperclip Orchestrator (The Boss / Control Plane)    │
│  LAYER 3 ── Hermes Main Agent + Subagents (The Workforce)        │
│  LAYER 4 ── Obsidian Vault + PostgreSQL (Memory + Storage)       │
│  LAYER 5 ── Skills, Routines, Governance, Budgets                │
│  LAYER 6 ── Monitoring, Alerting, Hardening, Go-Live             │
└──────────────────────────────────────────────────────────────────┘
```

PHASE MAP AT A GLANCE

#	Phase Name	Layer	Outcome
01	OS, Git & Folder Baseline	0	Clean repo, directory tree, Git history
02	Docker Infrastructure	0	PostgreSQL + pgAdmin + Redis running
03	Environment & Secrets Vault	0	`.env` hierarchy, secrets manager
04	Graphify Global Install & Scan	1	Full knowledge graph of all repos
05	Graphify MCP Service & Live Watch	1	Real-time graph updates via MCP
06	Paperclip Install & DB Isolation	2	Boss running on port 3100
07	Paperclip Company & Org Chart	2	Goals, roles, hierarchy defined

#	Phase Name	Layer	Outcome
08	Paperclip Budgets & Governance	2	Hard stops, approval gates, audit
09	Hermes Main Agent Install	3	Hermes TUI running, models set
10	Hermes ↔ Graphify Integration	3	Agent reads graph before every action
11	Hermes ↔ Paperclip Integration	3	Hermes hired into Paperclip org chart
12	Hermes Subagent Architecture	3	Parallel subagents spawned per task
13	Obsidian Vault Schema	4	Full vault structure, templates
14	PostgreSQL Schema & Tables	4	ALZEN relational DB, migrations
15	Hermes Skills — Sales Dept	5	Qualifier, Enrichment, Email skills
16	Hermes Skills — Ops Dept	5	Call Handler, Memory Manager skills
17	Paperclip Routines & Schedules	5	Cron heartbeats, event triggers
18	Telegram Mobile Control Layer	5	Approve/override from phone
19	Monitoring, Logging & Alerting	6	Grafana/logs, cost dashboards
20	Security Hardening & DR	6	Secrets rotation, backups, rollback
21	Production Go-Live & Validation	6	Full end-to-end smoke test, live

PHASE 01 — OS, Git & Folder Baseline

Goal

Establish a reproducible, version-controlled workspace from scratch. Every file, config, and secret has a home. Nothing is ad-hoc.

Why This Comes First

Without a clean directory contract, every tool installs into random locations, configs clash, and you cannot roll back. Git history is the safety net for all 21 phases.

- Windows 11 with WSL2 (Ubuntu 22.04) **or** native Ubuntu 22.04
- Node.js 20+ and pnpm 9.15+
- Python 3.11+ (via `uv`)
- Git 2.40+
- Docker Desktop 4.x

Step-by-Step

1.1 — Clean the workspace

```bash
# DESTRUCTIVE — only run on fresh setup
rm -rf D:/ALZEN
mkdir -p D:/ALZEN
cd D:/ALZEN
```

1.2 — Create the canonical directory tree

```
D:/ALZEN/
├── .git/                          # Git history
├── .env.example                   # Template — NEVER commit real .env
├── .gitignore
├── .graphifyignore
├── README.md
├── docker-compose.yml
├── paperclip/                     # Paperclip server source (cloned)
├── hermes/                        # Hermes agent source (cloned)
├── graphify/                      # Graphify outputs
|   ├── graph.html
|   ├── graph.json
|   └── GRAPH_REPORT.md
├── vault/                         # Obsidian markdown vault
|   ├── leads/
|   ├── enrichments/
|   ├── emails/
|   ├── calls/
|   ├── patterns/
|   ├── agents/
|   └── _templates/
```

```
├── scripts/                    # Utility bash/python scripts
├── skills/                     # Hermes custom skills
└── logs/                       # Runtime logs
```

```bash
mkdir -p vault/{leads,enrichments,emails,calls,patterns,agents,_templates}
mkdir -p graphify scripts skills logs
```

1.3 — Initialize Git

```bash
git init
git config user.name "ALZEN Operator"
git config user.email "allauddin1524@gmail.com"
```

1.4 — Create `.gitignore`

```gitignore
.env
.env.*
!.env.example
node_modules/
__pycache__/
*.pyc
venv/
.venv/
logs/*.log
graphify/cache/
vault/.obsidian/
*.key
*.pem
secrets/
```

1.5 — Create `.graphifyignore`

```
node_modules/
.git/
venv/
*.log
*.env
```

```
secrets/
graphify/cache/
```

1.6 — Initial commit

```bash
git add .
git commit -m "feat: phase-01 — workspace baseline established"
```

Deliverables

- Clean `D:/ALZEN` directory with full folder contract
- Git initialized with `.gitignore`
- `.graphifyignore` in place

PHASE 02 — Docker Infrastructure

Goal

Spin up the persistent data layer: PostgreSQL (ALZEN's relational brain), pgAdmin (visual DB UI), and Redis (future task queue caching). These run isolated from Paperclip's internal embedded DB.

Why Two Databases?

Paperclip manages its own embedded PostgreSQL on port `54329` for internal ticketing state. ALZEN's PostgreSQL on port `5432` is our data — leads, enrichments, operational tables. They must never collide.

Step-by-Step

2.1 — Create `docker-compose.yml`

```yaml
```

```yaml
version: "3.9"

services:
  postgres:
    image: postgres:16-alpine
    container_name: alzen_postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: alzen_db
      POSTGRES_USER: alzen_admin
      POSTGRES_PASSWORD: Hiking@786
    ports:
      - "5432:5432"
    volumes:
      - alzen_pg_data:/var/lib/postgresql/data
      - ./scripts/sql/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U alzen_admin -d alzen_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: alzen_pgadmin
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: allauddin1524@gmail.com
      PGADMIN_DEFAULT_PASSWORD: Hiking@786
    ports:
      - "5050:80"
    depends_on:
      postgres:
        condition: service_healthy

  redis:
    image: redis:7-alpine
    container_name: alzen_redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - alzen_redis_data:/data
```

```yaml
volumes:
  alzen_pg_data:
  alzen_redis_data:
```

2.2 — Create `scripts/sql/init.sql`

```sql
-- ALZEN bootstrap: extensions only. Tables created in Phase 14.
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

2.3 — Start containers

```bash
cd D:/ALZEN
docker compose up -d postgres pgadmin redis
```

2.4 — Verify

```bash
# Postgres health
docker exec alzen_postgres pg_isready -U alzen_admin -d alzen_db

# pgAdmin — open browser
# http://localhost:5050
# Login: allauddin1524@gmail.com / Hiking@786
# Add server: host=alzen_postgres, port=5432, db=alzen_db

# Redis ping
docker exec alzen_redis redis-cli ping
# Expected: PONG
```

2.5 — Commit

```bash
git add docker-compose.yml scripts/
git commit -m "feat: phase-02 — docker infrastructure (postgres, pgadmin, redis)"
```

- `alzen_postgres` container healthy on port 5432
- `alzen_pgadmin` accessible at localhost:5050
- `alzen_redis` responding on port 6379
- Init SQL with extensions

PHASE 03 — Environment & Secrets Management

Goal

Establish a layered `.env` system so every tool (Paperclip, Hermes, Graphify, scripts) reads credentials from a single source of truth — never hardcoded, never committed.

Architecture

```
.env.example          ← committed to Git (template, no values)
.env                  ← local only, gitignored (all real secrets)
.env.paperclip        ← Paperclip-specific overrides
.env.hermes           ← Hermes-specific overrides
scripts/load-env.sh   ← sources correct env file per tool
```

Step-by-Step

3.1 — Create `.env.example`

```bash
```

```
# ── ALZEN PostgreSQL ──────────────────────────────
ALZEN_DB_NAME=alzen_db
ALZEN_DB_USER=alzen_admin
ALZEN_DB_PASSWORD=
ALZEN_DB_HOST=localhost
ALZEN_DB_PORT=5432

# ── pgAdmin ───────────────────────────────────────
PGADMIN_EMAIL=
PGADMIN_PASSWORD=

# ── LLM Providers ─────────────────────────────────
ANTHROPIC_API_KEY=
OPENROUTER_API_KEY=
DEEPSEEK_API_KEY=
QWEN_API_KEY=

# ── Telegram Bot ──────────────────────────────────
TELEGRAM_BOT_TOKEN=
TELEGRAM_ALLOWED_USERS=

# ── Paperclip ─────────────────────────────────────
PAPERCLIP_BIND=loopback
PAPERCLIP_PORT=3100
PAPERCLIP_DB_PORT=54329

# ── Redis ─────────────────────────────────────────
REDIS_URL=redis://localhost:6379

# ── Graphify ──────────────────────────────────────
GRAPHIFY_OUTPUT_DIR=./graphify
GRAPHIFY_VAULT_DIR=./vault
```

3.2 — Create `.env` (populate all values)

```bash
cp .env.example .env
# Edit .env with real credentials
# NEVER commit this file
```

3.3 — Create `scripts/load-env.sh`

```bash
#!/bin/bash
# Usage: source scripts/load-env.sh [paperclip|hermes|base]
ENV_TARGET=${1:-base}
set -a
[ -f .env ] && source .env
[ -f .env.$ENV_TARGET ] && source .env.$ENV_TARGET
set +a
echo "✅ Environment loaded: $ENV_TARGET"
```

3.4 — Create `.env.paperclip`

```bash
DATABASE_URL=postgresql://localhost:54329/paperclip_internal
NODE_ENV=production
PORT=3100
```

3.5 — Create `.env.hermes`

```bash
HERMES_WORKING_DIR=D:/ALZEN
HERMES_VAULT_DIR=D:/ALZEN/vault
HERMES_SKILLS_DIR=D:/ALZEN/skills
```

3.6 — Commit templates only

```bash
git add .env.example .env.paperclip .env.hermes scripts/load-env.sh
git commit -m "feat: phase-03 — env & secrets layer (templates only)"
```

Deliverables

- Layered `.env` system
- No real secrets in Git
- `load-env.sh` utility script

PHASE 04 — Graphify: Global Install & Full Codebase Scan

Goal

Install Graphify and run it against the ENTIRE `D:/ALZEN` workspace — including Paperclip source, Hermes source, vault markdown, and scripts — to produce a unified knowledge graph that any agent can query before acting.

Why Graphify at the Top

Without a knowledge graph, agents blindly grep files, miss cross-repo connections, and waste tokens on redundant context loading. Graphify gives every agent a pre-built map with a 71.5× token reduction on queries.

Step-by-Step

4.1 — Install Graphify

```bash
pip install --user graphifyy
graphify install
# Verifies Tree-sitter, NetworkX, Leiden dependencies
```

4.2 — Clone Paperclip and Hermes repos

```bash
cd D:/ALZEN
git clone https://github.com/paperclipai/paperclip.git paperclip
git clone https://github.com/NousResearch/hermes-agent.git hermes
```

4.3 — Run the initial global scan

```bash
```

```
# This scans: paperclip/, hermes/, vault/, skills/, scripts/
graphify ./

# Options for production-quality scan
graphify ./ \
  --output-dir ./graphify \
  --include-extensions .py,.js,.ts,.md,.json,.yaml,.yml,.sql \
  --ignore-file .graphifyignore
```

This produces:

```
graphify/
├── graph.html          ← Interactive visual browser
├── graph.json          ← Machine-queryable graph
├── GRAPH_REPORT.md     ← Human-readable audit: god nodes, surprises
└── cache/              ← Incremental cache (future scans are fast)
```

4.4 — Review GRAPH_REPORT.md

Open `graphify/GRAPH_REPORT.md` and look for:

- **God Nodes** — highest-degree concepts (these are your system's critical paths)
- **Surprise Edges** — unexpected cross-file connections
- **Community Clusters** — natural module groupings

Expected god nodes in ALZEN:

- `HermesAgent`, `PaperclipOrchestrator`, `GraphifyScanner`
- `LeadQualifier`, `vault/patterns/icp-signals.md`
- `alzen_db`, `docker-compose`

4.5 — Open interactive graph

```bash
# Start local HTTP server to view graph.html
python -m http.server 8888 --directory ./graphify
# Open: http://localhost:8888/graph.html
```

4.6 — Configure Graphify for Claude Code integration

Create `CLAUDE.md` in project root:

```markdown
# ALZEN Project — Claude Code Directives

## Graph Navigation
Before exploring any file, run:
/graphify query "<concept>"

## Project Structure
- paperclip/ — Orchestrator (Node.js/React)
- hermes/ — Agent workforce (Python)
- vault/ — Obsidian markdown memory
- graphify/ — Knowledge graph outputs

## Critical Paths (God Nodes)
See graphify/GRAPH_REPORT.md for current god nodes.
Always check graph before grepping files.
```

4.7 — Commit

```bash
git add graphify/GRAPH_REPORT.md CLAUDE.md .graphifyignore
git commit -m "feat: phase-04 — graphify initial global scan complete"
```

Deliverables

- Graphify installed globally
- `graphify/graph.html` — interactive knowledge map
- `graphify/GRAPH_REPORT.md` — god nodes and surprises documented
- `CLAUDE.md` with graph navigation directives

PHASE 05 — Graphify: MCP Service & Live Watch Mode

Goal

Run Graphify as a persistent MCP (Model Context Protocol) server so Hermes agents can query the knowledge graph in real-time as a tool call — not just at scan time. Enable file watcher so the

graph auto-updates when code changes.

Architecture

```
[Hermes Agent] ──tool call──▶ [Graphify MCP Server :8889]
                                        |
                                reads graph.json
                                        |
                                returns: nodes, paths, god nodes
```

Step-by-Step

5.1 — Start Graphify as MCP server

```bash
# Start the MCP service (persists across sessions)
graphify serve --port 8889 --graph-path ./graphify/graph.json

# Or as a background daemon
nohup graphify serve --port 8889 --graph-path ./graphify/graph.json \
  > logs/graphify-mcp.log 2>&1 &
echo $! > logs/graphify-mcp.pid
```

5.2 — Enable live file watch mode

```bash
# In a second terminal — watches for file changes and rebuilds graph
graphify watch ./ \
  --output-dir ./graphify \
  --ignore-file .graphifyignore \
  --debounce 30s \
  > logs/graphify-watch.log 2>&1 &
echo $! > logs/graphify-watch.pid
```

5.3 — Install Graphify into Hermes

```bash
```

```
# This registers Graphify as a native tool inside Hermes
graphify hermes install

# Verify
hermes tools | grep graphify
```

After this, Hermes can use:

```
/graphify query "lead qualification logic"
/graphify path hermes/skills/ vault/patterns/
/graphify explain HermesAgent
/graphify god-nodes --top 10
```

5.4 — Create `scripts/start-graphify.sh`

```bash
#!/bin/bash
source scripts/load-env.sh base

echo "🔍 Starting Graphify MCP server..."
graphify serve --port 8889 --graph-path ./graphify/graph.json \
  > logs/graphify-mcp.log 2>&1 &
echo $! > logs/graphify-mcp.pid

echo "👁️  Starting Graphify watch mode..."
graphify watch ./ \
  --output-dir ./graphify \
  --ignore-file .graphifyignore \
  --debounce 30s \
  > logs/graphify-watch.log 2>&1 &
echo $! > logs/graphify-watch.pid

echo "✅ Graphify running — MCP: http://localhost:8889"
```

5.5 — Add to Hermes config as MCP server

Edit `~/.hermes/config.yaml`:

```yaml
```

```yaml
mcp_servers:
  - name: alzen-knowledge-graph
    url: http://localhost:8889/mcp
    description: "ALZEN full codebase knowledge graph — query before acting"
    always_load: true
```

5.6 — Test MCP endpoint

```bash
curl http://localhost:8889/mcp/god-nodes?top=5
# Expected JSON: [{node: "HermesAgent", degree: 42}, ...]
```

5.7 — Commit

```bash
git add scripts/start-graphify.sh
git commit -m "feat: phase-05 — graphify MCP service + live watch enabled"
```

Deliverables

- Graphify MCP server on port 8889
- Live watch daemon auto-rebuilding graph on file changes
- Hermes registered to query graph via MCP tool calls
- `start-graphify.sh` startup script

PHASE 06 — Paperclip: Install & DB Isolation

Goal

Install Paperclip (The Boss) as the company orchestrator. Critically isolate its embedded PostgreSQL from ALZEN's operational database so they never conflict.

Step-by-Step

6.1 — Install Paperclip via npx onboard

```bash
```

```
cd D:/ALZEN

# Default loopback mode (fastest, local only)
npx paperclipai onboard --yes

# For access from phone via Tailscale:
# npx paperclipai onboard --yes --bind tailnet
```

6.2 — Verify embedded DB isolation

```bash
# Paperclip's DB must be on 54329, NOT 5432
npx paperclipai configure

# In the config menu, confirm:
# Database → Embedded PostgreSQL
# Port → 54329
# This port must NOT conflict with alzen_postgres on 5432
```

6.3 — Start Paperclip

```bash
npx paperclipai run
# Or if cloned manually:
cd D:/ALZEN/paperclip
pnpm install
pnpm dev
```

6.4 — Verify dashboard

Open: `http://localhost:3100` Expected: Paperclip UI loads with empty company/no agents.

6.5 — Create `scripts/start-paperclip.sh`

```bash
```

```
#!/bin/bash
source scripts/load-env.sh paperclip

echo "🏢 Starting Paperclip orchestrator..."
cd D:/ALZEN/paperclip
pnpm dev > ../logs/paperclip.log 2>&1 &
echo $! > ../logs/paperclip.pid

echo "✅ Paperclip running — Dashboard: http://localhost:3100"
```

6.6 — Confirm port separation

```bash
# Should show TWO postgres processes on DIFFERENT ports
netstat -tlnp | grep postgres
# Expected:
# 0.0.0.0:5432  (alzen_postgres - Docker)
# 127.0.0.1:54329 (paperclip embedded)
```

6.7 — Commit

```bash
git add scripts/start-paperclip.sh
git commit -m "feat: phase-06 — paperclip installed, db isolation verified"
```

Deliverables

- Paperclip UI live at `http://localhost:3100`
- Embedded DB locked to port 54329
- Zero conflict with ALZEN PostgreSQL on 5432

PHASE 07 — Paperclip: Company, Goals & Org Chart

Goal

Build the ALZEN company structure inside Paperclip — the mission, org chart, agent roles, and goal hierarchy. This is the management layer that coordinates all downstream work.

Step-by-Step

7.1 — Create the Company

In Paperclip UI → Companies → New Company:

```
Company Name:    ALZEN Autonomous Sales Agency
Mission:         Qualify, enrich, and close B2B leads at scale with zero human
bottlenecks
Industry:        B2B SaaS Sales
Monthly Budget:  $500 USD (token cost cap)
```

7.2 — Define Top-Level Goal

Goals → New Goal:

```
Goal:     Qualify and email 500 leads per month achieving >15% reply rate
Parent:   (Company Mission)
Success:  500 leads processed, >15% positive reply, <$0.10 cost per lead
Timeline: 30 days
```

7.3 — Create Sub-Goals

```
Sub-Goal 1: Lead Qualification
  → Score 100% of incoming leads via DeepSeek within 1 hour of receipt

Sub-Goal 2: Lead Enrichment
  → Deep-research all qualified leads, extract top 3 pain points

Sub-Goal 3: Email Outreach
  → Write and send 2 A/B variant emails per lead within 4 hours of enrichment

Sub-Goal 4: Reply Handling
  → Categorize all replies within 30 minutes, escalate objections to Telegram

Sub-Goal 5: Pattern Learning
  → Weekly vault analysis to improve ICP signals and email templates
```

7.4 — Build the Org Chart

Org Chart → Add Agents:

Title	Role	Reports To	Budget/mo
ALZEN CEO	Strategy, goal decomposition	Board (You)	$50
Sales Director	Coordinates sales pipeline	CEO	$30
Lead Qualifier	Scores incoming leads	Sales Director	$80
Enrichment Analyst	Deep research per lead	Sales Director	$100
Email Writer	Drafts outreach emails	Sales Director	$100
Call Handler	Processes webhook replies	Sales Director	$40
Memory Manager	Weekly pattern analysis	CEO	$30
Graphify Scanner	Knowledge graph updates	CEO	$20

7.5 — Set Reporting Lines

```
Board (You)
  └── CEO (ALZEN)
        ├── Sales Director
        |     ├── Lead Qualifier
        |     ├── Enrichment Analyst
        |     ├── Email Writer
        |     └── Call Handler
        ├── Memory Manager
        └── Graphify Scanner
```

7.6 — Commit config export

```bash
# Export org structure for version control
npx paperclipai export --company ALZEN > scripts/alzen-company-export.json
git add scripts/alzen-company-export.json
git commit -m "feat: phase-07 — paperclip company, goals, org chart defined"
```

Deliverables

- ALZEN company created in Paperclip
- Mission and 5 sub-goals defined with success metrics

- 8-agent org chart with budgets and reporting lines
- Org exported to version-controlled JSON

PHASE 08 — Paperclip: Budgets, Governance & Approval Gates

Goal

Configure hard budget limits, governance policies, and approval gates so no agent can spend unbounded tokens or take irreversible actions without board sign-off (you).

Step-by-Step

8.1 — Set Company-Wide Budget Policy

Paperclip → Company → Budget:

```
Monthly Limit:      $500
Warning Threshold:  80% ($400)
Hard Stop:          100% ($500) → all agents paused automatically
Reset Day:          1st of each month
Currency:           USD
Token Tracking:     Per agent, per model, per task
```

8.2 — Set Per-Agent Budgets (Enforcement)

Agent	Monthly Cap	Warning At	Action at Cap
CEO	$50	$40	Pause + Telegram alert
Lead Qualifier	$80	$64	Pause + queue backlog
Enrichment Analyst	$100	$80	Pause + notify Sales Director
Email Writer	$100	$80	Pause + notify
Call Handler	$40	$32	Pause
Memory Manager	$30	$24	Pause
Graphify Scanner	$20	$16	Pause

8.3 — Define Approval Gates (Governance)

Paperclip → Governance → New Policy:

Gate 1: Email Send

```
Trigger:    Agent attempts to send any outbound email
Required:   Board approval OR Sales Director approval
Timeout:    2 hours (auto-approve if no response, first 7 days: manual only)
Rollback:   Cancel scheduled send
```

Gate 2: New Agent Hire

```
Trigger:    Any new agent added to org chart
Required:   Board approval only (you)
Timeout:    No timeout — must be manual
```

Gate 3: Budget Override Request

```
Trigger:    Agent requests budget increase
Required:   Board approval
Timeout:    No timeout
```

Gate 4: Skill Modification

```
Trigger:    Hermes autonomously modifies an existing skill
Required:   Sales Director review within 24h
Timeout:    24h (auto-approve if positive track record)
```

Gate 5: External API Call > 100 requests/hour

```
Trigger:    Agent attempts high-frequency external API calls
Required:   Board approval
Timeout:    1 hour
```

8.4 — Enable Audit Logging

```
Paperclip → Settings → Audit Log → Enable:
  ✅ All task state changes
  ✅ All budget events
  ✅ All governance decisions
  ✅ All tool calls (full trace)
```

```
  ✅ All approval gate outcomes
Retention: 90 days
Export: Weekly to vault/agents/audit-logs/
```

8.5 — Test governance enforcement

```bash
# Simulate an over-budget run (test environment)
# In Paperclip UI → Agent → Lead Qualifier → Simulate Budget Exceeded
# Expected: Agent pauses, Telegram notification fires, dashboard shows "BUDGET LIMIT
```

8.6 — Commit governance config

```bash
npx paperclipai export --company ALZEN --include-governance > scripts/alzen-governan
git add scripts/alzen-governance.json
git commit -m "feat: phase-08 — budgets, governance gates, audit logging configured"
```

Deliverables

- $500/month hard cap with per-agent limits
- 5 approval gates covering emails, hires, budgets, skills, API volume
- Full audit logging to vault
- Tested: budget enforcement pauses agents automatically

PHASE 09 — Hermes: Main Agent Install & Model Configuration

Goal

Install Hermes (The Workforce) globally, configure it to use DeepSeek for reasoning/scoring and Qwen for email writing, and set the working directory to `D:/ALZEN`.

9.1 — Install via uv (recommended)

```bash
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv ~/.hermes-venv --python 3.11
source ~/.hermes-venv/bin/activate
uv pip install "hermes-agent[all]"

# Symlink to PATH
ln -sf ~/.hermes-venv/bin/hermes ~/.local/bin/hermes
```

9.2 — Run initial setup wizard

```bash
hermes setup
```

In the wizard, configure:

```
Working Directory:    D:/ALZEN
Default Model:        deepseek/deepseek-r1        (reasoning/scoring)
Email Model:          qwen/qwen2.5-72b-instruct   (email writing)
Provider:             OpenRouter (supports both models)
OpenRouter API Key:   [your key from .env]
Enable Telegram:      Yes (configure in Phase 18)
Enable Skills:        Yes
Memory:               Enabled
```

9.3 — Verify installation

```bash
hermes doctor
# Expected: All checks green

hermes model
# Expected: Shows deepseek/deepseek-r1 as active
```

9.4 — Configure SOUL.md (Agent Persona)

Create `~/.hermes/SOUL.md`:

```markdown
# ALZEN Agent Persona

You are the ALZEN autonomous sales agent. You operate inside the ALZEN B2B
sales agency on behalf of the operator.

## Core Directives
1. Always query the Graphify knowledge graph BEFORE reading any file
2. Write every reasoning step and decision to the vault
3. Score leads ruthlessly — protect token budget, block low-quality leads
4. When uncertain, write a blocker ticket in Paperclip rather than guessing
5. Escalate to Telegram on any objection, unresolved blocker, or budget warning

## Model Routing
- Reasoning, scoring, analysis: deepseek/deepseek-r1
- Email writing, copy: qwen/qwen2.5-72b-instruct
- Fast classification: any cheap model via OpenRouter

## Vault First
All reasoning is written to vault/ before any external action.
```

9.5 — Set Hermes working directory

```bash
hermes config set working_directory D:/ALZEN
hermes config set vault_dir D:/ALZEN/vault
hermes config set skills_dir D:/ALZEN/skills
```

9.6 — Test first conversation

```bash
hermes
# Type: "Summarize the ALZEN project structure using the knowledge graph"
# Expected: Hermes queries Graphify MCP, returns structured summary
```

9.7 — Commit

```bash
```

```
git add ~/.hermes/SOUL.md
git commit -m "feat: phase-09 — hermes installed, deepseek+qwen configured, soul def
```

Deliverables

- Hermes running with `hermes doctor` all-green
- DeepSeek (reasoning) + Qwen (email) model routing
- SOUL.md persona with ALZEN directives
- Working directory set to `D:/ALZEN`

PHASE 10 — Hermes ↔ Graphify Integration

Goal

Wire Hermes's PreToolUse hook to automatically query the Graphify knowledge graph before any file operation, ensuring agents always navigate by map, not by blind grep.

Architecture

```
Hermes receives task
    |
    ▼
PreToolUse Hook fires
    |
    ▼
/graphify query "<task context>"
    |
    ▼
Returns: relevant nodes, paths, god nodes
    |
    ▼
Hermes uses graph context in prompt
    |
    ▼
Executes actual tool call
```

Step-by-Step

10.1 — Register Graphify MCP in Hermes config

Edit `~/.hermes/config.yaml`:

```yaml
mcp_servers:
  - name: alzen-graphify
    url: http://localhost:8889/mcp
    description: "ALZEN knowledge graph — always query first"
    always_load: true
    pre_tool_use: true   # Run before every tool call
```

10.2 — Create PreToolUse hook skill

Create `D:/ALZEN/skills/graphify-pre-hook.md`:

```markdown
# Skill: Graphify Pre-Hook

## Trigger
Before ANY file read, write, or search operation.

## Steps
1. Extract the core concept from the current task
2. Run: /graphify query "<concept>"
3. Review returned nodes and paths
4. Use the top 3 most-connected nodes as primary context
5. Proceed with the original tool call

## Why
The Graphify graph has a 71.5x token reduction vs raw file scanning.
Always navigate by map first.

## Skip conditions
- Simple arithmetic or pure text generation (no file access needed)
- Already have graph context from this same task turn
```

10.3 — Verify integration

```bash
```

```
hermes
# Type: "What is the lead qualification logic?"
# Expected output sequence:
# [Graphify MCP called] → [Returns: LeadQualifier node, icp-signals.md path]
# [Agent uses graph context] → [Answers without reading raw files]
```

10.4 — Add graph query shortcut to CLAUDE.md

```markdown
## Graphify MCP Commands (available in Hermes)
/graphify query "<concept>"        — semantic search on graph
/graphify path <file1> <file2>     — shortest path between concepts
/graphify explain <node>           — full node context
/graphify god-nodes --top 10       — show highest-degree nodes
/graphify surprises                — show unexpected cross-connections
```

10.5 — Commit

```bash
git add skills/graphify-pre-hook.md CLAUDE.md
git commit -m "feat: phase-10 — hermes pre-tool-use graphify hook integrated"
```

Deliverables

- Hermes PreToolUse hook queries Graphify before every file operation
- Graph MCP registered as `always_load: true`
- 71.5× token reduction active for agent file navigation
- Pre-hook skill documented in `skills/`

PHASE 11 — Hermes ↔ Paperclip Integration

Goal

Formally hire Hermes into Paperclip's org chart as the executable agent workforce. Paperclip assigns tasks via heartbeats/webhooks; Hermes executes and reports back with structured results.

Integration Architecture

```
Paperclip (port 3100)
    |
    |── Heartbeat: every 15 min ──▶ Hermes CLI adapter
    |                                        |
    |                              picks up pending tasks
    |                                        |
    |                              executes with DeepSeek/Qwen
    |                                        |
    |◀─── writes result ──────────── vault/ + postgres update
    |
    ▲── Board approvals via Telegram
```

Step-by-Step

11.1 — Create Hermes Heartbeat Adapter Script

Create `scripts/hermes-heartbeat.sh`:

```bash
```

```bash
#!/bin/bash
# Called by Paperclip on each heartbeat for this agent
source D:/ALZEN/scripts/load-env.sh hermes

TASK_ID=$1
TASK_TITLE=$2
TASK_CONTEXT=$3

echo "💗 Heartbeat received: Task $TASK_ID — $TASK_TITLE"

# Execute task via Hermes non-interactive mode
hermes run --task "$TASK_TITLE" \
           --context "$TASK_CONTEXT" \
           --task-id "$TASK_ID" \
           --output-format json \
           --write-vault \
  >> D:/ALZEN/logs/hermes-tasks.log 2>&1

EXIT_CODE=$?

# Report back to Paperclip
if [ $EXIT_CODE -eq 0 ]; then
  echo "✅ Task $TASK_ID completed"
  curl -s -X POST http://localhost:3100/api/tasks/$TASK_ID/complete \
    -H "Content-Type: application/json" \
    -d '{"status": "done", "agent": "hermes"}'
else
  echo "❌ Task $TASK_ID failed"
  curl -s -X POST http://localhost:3100/api/tasks/$TASK_ID/fail \
    -H "Content-Type: application/json" \
    -d '{"status": "failed", "agent": "hermes"}'
fi
```

11.2 — Register Hermes as Agent in Paperclip UI

Paperclip → Org Chart → Lead Qualifier → Edit Agent:

```
Agent Name:       Hermes Lead Qualifier
Adapter Type:     CLI (bash script)
Heartbeat Command: bash D:/ALZEN/scripts/hermes-heartbeat.sh
Heartbeat Interval: 15 minutes
Working Directory:  D:/ALZEN
Environment File:   D:/ALZEN/.env.hermes
Model:              deepseek/deepseek-r1
```

Repeat for each agent role (Enrichment, Email Writer, Call Handler, Memory Manager).

11.3 — Configure task context injection

In Paperclip → Agent Settings → Context:

```yaml
context_template: |
  Company Goal: {company.goal}
  Project: {project.name}
  Parent Task: {task.parent_title}
  Task: {task.title}
  Description: {task.description}
  Due: {task.due_date}
  Graph Report: {graphify.god_nodes}
  ICP Signals: {vault.patterns.icp-signals}
```

11.4 — Test first heartbeat

```bash
# In Paperclip UI → Create test task → Assign to Hermes Lead Qualifier
# Task: "Score lead: Acme Corp, Series B, 200 employees, SaaS"
# Watch hermes-tasks.log for execution trace
tail -f logs/hermes-tasks.log
```

11.5 — Commit

```bash
git add scripts/hermes-heartbeat.sh
git commit -m "feat: phase-11 — hermes hired into paperclip via CLI adapter"
```

Deliverables

- `hermes-heartbeat.sh` CLI adapter script
- All 7 agent roles registered in Paperclip
- Context template injects goal ancestry into every task
- First heartbeat test passes

PHASE 12 — Hermes: Subagent Architecture for Parallelism

Goal

Configure Hermes to spawn isolated subagents for parallel workstreams — e.g., enriching 50 leads simultaneously rather than sequentially. Each subagent runs in an isolated workspace with its own context.

Architecture

```
Hermes Main (Enrichment Analyst role)
    |
    ├── spawn subagent: lead-001-enrich
    ├── spawn subagent: lead-002-enrich
    ├── spawn subagent: lead-003-enrich
    |       ... (up to N parallel workers)
    |
    ▼ (all complete)
Main collects results → writes to vault/ → reports to Paperclip
```

Step-by-Step

12.1 — Create subagent spawn script

Create `scripts/spawn-subagents.sh`:

```bash
```

```bash
#!/bin/bash
# Usage: bash spawn-subagents.sh <task_type> <input_file> <concurrency>
source D:/ALZEN/scripts/load-env.sh hermes

TASK_TYPE=$1      # e.g., "enrich", "qualify", "email"
INPUT_FILE=$2     # CSV or JSON of items to process
CONCURRENCY=${3:-5}  # Max parallel workers (default 5)

echo "🚀 Spawning $CONCURRENCY subagents for: $TASK_TYPE"

# Read each line of input, spawn Hermes subagent per item
while IFS= read -r item; do
  (
    hermes run \
      --subagent \
      --task "$TASK_TYPE: $item" \
      --isolation full \
      --write-vault \
      --timeout 300
  ) &

  # Limit concurrency
  while [ $(jobs -r | wc -l) -ge $CONCURRENCY ]; do
    sleep 2
  done

done < "$INPUT_FILE"

# Wait for all subagents to complete
wait
echo "✅ All subagents completed for $TASK_TYPE"
```

12.2 — Configure subagent isolation in Hermes

Edit `~/.hermes/config.yaml`:

```yaml
```

```yaml
subagents:
  enabled: true
  isolation: full              # Each subagent has its own context
  max_concurrent: 10           # Hard cap
  workspace_mode: ephemeral    # Clean workspace per subagent
  inherit_mcp: true            # Subagents also get Graphify access
  inherit_skills: true
  write_results_to: vault/     # All subagent output goes to vault
  cost_tracking: per_subagent  # Budget tracked individually
```

12.3 — Create parallel enrichment skill

Create `D:/ALZEN/skills/parallel-enrichment.md`:

```markdown
# Skill: Parallel Lead Enrichment

## Trigger
When a batch of leads (>5) needs enrichment simultaneously.

## Steps
1. Read lead IDs from vault/leads/ directory
2. Chunk into batches of 5
3. For each batch: spawn subagents via spawn-subagents.sh
4. Monitor subagent completion (check vault/enrichments/ for outputs)
5. Collect all results once complete
6. Write summary to vault/patterns/batch-{date}.md
7. Update PostgreSQL lead_status table

## Concurrency Rules
- Max 5 subagents simultaneously (cost control)
- Each subagent budget: $0.05 max
- Abort subagent if it exceeds budget

## Failure Handling
- If subagent fails: write to vault/enrichments/{lead-id}-FAILED.md
- Do NOT retry automatically — escalate to Paperclip blocker
```

12.4 — Test subagent parallelism

```bash
```

```bash
# Create test batch of 10 lead IDs
echo -e "lead-001\nlead-002\nlead-003\nlead-004\nlead-005" > /tmp/test-leads.txt

# Spawn 5 concurrent subagents
bash scripts/spawn-subagents.sh enrich /tmp/test-leads.txt 5

# Monitor outputs
watch -n 2 "ls -la vault/enrichments/"
```

12.5 — Commit

```bash
git add scripts/spawn-subagents.sh skills/parallel-enrichment.md
git commit -m "feat: phase-12 — hermes subagent parallelism configured"
```

Deliverables

- `spawn-subagents.sh` script for batch parallel processing
- Subagent isolation config in Hermes
- `parallel-enrichment.md` skill
- Tested: 5 concurrent subagents executing and writing to vault

---

PHASE 13 — Obsidian Vault Schema & Templates

Goal

Define the complete Obsidian vault structure with YAML frontmatter schemas and markdown templates. The vault is the company's human-readable memory — every agent reads from and writes to it.

Vault Directory Contract

```
vault/
├── leads/              — Raw lead intake files
├── enrichments/        — Deep research per lead
├── emails/             — Generated email drafts and variants
```

```
├── calls/                — Webhook reply categorizations
├── patterns/             — Learned ICP signals and patterns
├── agents/               — Agent logs, decisions, audit trails
|   └── audit-logs/
└── _templates/           — Blank templates per type
```

Step-by-Step

13.1 — Lead Template

Create `vault/_templates/lead.md`:

```markdown
---
lead_id: "{{lead_id}}"
company: "{{company_name}}"
contact: "{{contact_name}}"
email: "{{email}}"
title: "{{title}}"
linkedin: "{{linkedin_url}}"
company_size: 0
funding_stage: ""
industry: ""
source: ""
date_received: "{{date}}"
status: "new"          # new | qualified | enriched | emailed | replied | closed
qualification_score: 0  # 1-10
disqualified_reason: ""
---

# {{company_name}} — Lead File

## Raw Data
<!-- Paste raw lead data here -->

## Qualification Notes
<!-- Filled by Lead Qualifier agent -->

## ICP Match Signals
<!-- Which signals from icp-signals.md matched -->
```

13.2 — Enrichment Template

Create `vault/_templates/enrichment.md`:

```markdown
---
lead_id: "{{lead_id}}"
enriched_at: "{{date}}"
model_used: "deepseek/deepseek-r1"
cost_usd: 0.00
confidence: 0          # 1-10
---

# {{company_name}} — Enrichment Report

## Company Context
<!-- 2-3 sentence company overview -->

## Top 3 Pain Points
1.
2.
3.

## Buying Signals
<!-- Evidence of purchase intent -->

## Decision Maker Analysis
<!-- Who holds the budget? Who influences? -->

## Recommended Email Angle
<!-- Which pain point to lead with -->

## Sources Consulted
<!-- URLs, Graphify nodes, vault files referenced -->
```

13.3 — Email Draft Template

Create `vault/_templates/email.md`:

```markdown
```

```
---
lead_id: "{{lead_id}}"
variant: "A"            # A | B
model_used: "qwen/qwen2.5-72b-instruct"
status: "draft"         # draft | approved | sent | replied
sent_at: ""
opened: false
replied: false
reply_sentiment: ""     # positive | objection | not_now | no_reply
---

# Email Draft — {{company_name}} — Variant {{variant}}

## Subject
{{subject_line}}

## Body
{{email_body}}

## Send Notes
<!-- Any timing, personalization, or sequencing notes -->
```

13.4 — ICP Signals Pattern File

Create `vault/patterns/icp-signals.md`:

```markdown
```

```
---
last_updated: "{{date}}"
updated_by: "memory-manager-agent"
---

# ICP Signals — Ideal Customer Profile

## Strong Positive Signals (qualify immediately)
- Series A-C funding within last 12 months
- 50-500 employees
- SaaS product with sales team
- Growing headcount (LinkedIn +20% YoY)
- Job postings for SDRs or AEs

## Soft Positive Signals (qualify with caution)
- Bootstrapped but profitable
- 500-2000 employees (may be too large)
- Multiple decision makers visible on LinkedIn

## Disqualification Signals (skip immediately — save tokens)
- <10 employees (too small, no budget)
- >5000 employees (too large, long cycle)
- Non-SaaS (wrong fit)
- Government or education (procurement hell)
- No visible online presence

## Historical Pattern Notes
<!-- Filled weekly by Memory Manager agent -->
```

13.5 — Agent Decision Log Template

Create `vault/_templates/agent-decision.md`:

```markdown
```

```
---
agent: "{{agent_name}}"
task_id: "{{paperclip_task_id}}"
date: "{{date}}"
decision_type: ""      # qualify | enrich | email | route | escalate
outcome: ""            # completed | blocked | escalated | failed
cost_usd: 0.00
tokens_used: 0
---

# Agent Decision Log

## Task Context
<!-- What was the task? What was the goal? -->

## Graph Nodes Consulted
<!-- Which Graphify nodes informed this decision? -->

## Reasoning
<!-- Step-by-step reasoning trace -->

## Decision
<!-- What did the agent decide and why? -->

## Output Written
<!-- What files were created/modified? -->

## Escalations
<!-- Any blockers filed in Paperclip? Telegram alerts sent? -->
```

13.6 — Initialize vault with starter files

```bash
cp vault/_templates/lead.md vault/leads/EXAMPLE-lead.md
cp vault/_templates/enrichment.md vault/enrichments/EXAMPLE-enrichment.md
cp vault/_templates/email.md vault/emails/EXAMPLE-email.md
```

13.7 — Commit vault schema

```bash
```

```
git add vault/
git commit -m "feat: phase-13 — obsidian vault schema, templates, icp-signals define
```

Deliverables

- 6 vault directories with clear purpose
- 5 markdown templates with YAML frontmatter
- `icp-signals.md` seeded with qualification rules
- `agent-decision.md` template for full audit trail

PHASE 14 — PostgreSQL Schema & Table Migrations

Goal

Define ALZEN's relational database schema — the persistent operational state that complements the Obsidian vault. Vault = human-readable reasoning; PostgreSQL = machine-queryable state.

Step-by-Step

14.1 — Create migration directory

```bash
mkdir -p scripts/sql/migrations
```

14.2 — Migration 001: Core tables

Create `scripts/sql/migrations/001_core_tables.sql`:

```sql
```

```sql
-- ==============================================================
-- ALZEN V2.0 — Core Schema
-- ==============================================================

-- Leads table
CREATE TABLE IF NOT EXISTS leads (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  company       VARCHAR(255) NOT NULL,
  contact_name  VARCHAR(255),
  email         VARCHAR(255) UNIQUE,
  title         VARCHAR(255),
  linkedin_url  TEXT,
  company_size  INTEGER,
  funding_stage VARCHAR(50),
  industry      VARCHAR(100),
  source        VARCHAR(100),
  status        VARCHAR(50) DEFAULT 'new'
                  CHECK (status IN ('new','qualifying','qualified',
                                    'disqualified','enriching','enriched',
                                    'emailed','replied','closed')),
  qual_score    SMALLINT CHECK (qual_score BETWEEN 0 AND 10),
  disqual_reason TEXT,
  vault_path    TEXT,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  updated_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Enrichments table
CREATE TABLE IF NOT EXISTS enrichments (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  lead_id       UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
  pain_points   JSONB,          -- Array of strings
  buying_signals JSONB,
  recommended_angle TEXT,
  model_used    VARCHAR(100),
  cost_usd      NUMERIC(8,4),
  tokens_used   INTEGER,
  confidence    SMALLINT CHECK (confidence BETWEEN 0 AND 10),
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Emails table
CREATE TABLE IF NOT EXISTS emails (
```

```sql
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  lead_id       UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
  variant       CHAR(1) CHECK (variant IN ('A','B','C')),
  subject       TEXT,
  body          TEXT,
  model_used    VARCHAR(100),
  status        VARCHAR(50) DEFAULT 'draft'
                  CHECK (status IN ('draft','approved','sent','replied','bounced')),
  sent_at       TIMESTAMPTZ,
  opened        BOOLEAN DEFAULT FALSE,
  replied       BOOLEAN DEFAULT FALSE,
  reply_text    TEXT,
  reply_sentiment VARCHAR(50),  -- positive | objection | not_now | no_reply
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Agent events table (cost & activity tracking)
CREATE TABLE IF NOT EXISTS agent_events (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  agent_name    VARCHAR(100) NOT NULL,
  task_id       VARCHAR(255),   -- Paperclip task ID
  event_type    VARCHAR(100),   -- qualify | enrich | email | heartbeat | escalate
  lead_id       UUID REFERENCES leads(id),
  model_used    VARCHAR(100),
  cost_usd      NUMERIC(8,4),
  tokens_used   INTEGER,
  outcome       VARCHAR(50),    -- completed | failed | blocked
  notes         TEXT,
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Weekly patterns table
CREATE TABLE IF NOT EXISTS patterns (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  week_start    DATE NOT NULL,
  pattern_type  VARCHAR(100),   -- icp_signal | email_angle | timing
  description   TEXT,
  confidence    NUMERIC(4,2),
  sample_size   INTEGER,
  created_by    VARCHAR(100),   -- agent name
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Indexes
```

```sql
CREATE INDEX idx_leads_status ON leads(status);
CREATE INDEX idx_leads_email ON leads(email);
CREATE INDEX idx_agent_events_agent ON agent_events(agent_name, created_at);
CREATE INDEX idx_agent_events_lead ON agent_events(lead_id);
CREATE INDEX idx_emails_lead ON emails(lead_id);
```

14.3 — Migration 002: Auto-update trigger

Create `scripts/sql/migrations/002_triggers.sql`:

```sql
-- Auto-update updated_at on leads
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER leads_updated_at
  BEFORE UPDATE ON leads
  FOR EACH ROW EXECUTE FUNCTION update_updated_at();

-- Agent cost summary view
CREATE OR REPLACE VIEW agent_cost_summary AS
SELECT
  agent_name,
  DATE_TRUNC('month', created_at) AS month,
  COUNT(*)                        AS total_events,
  SUM(tokens_used)                AS total_tokens,
  SUM(cost_usd)                   AS total_cost_usd,
  AVG(cost_usd)                   AS avg_cost_per_event
FROM agent_events
GROUP BY agent_name, DATE_TRUNC('month', created_at)
ORDER BY month DESC, total_cost_usd DESC;
```

14.4 — Run migrations

```bash
```

```bash
# Apply all migrations
docker exec -i alzen_postgres psql -U alzen_admin -d alzen_db \
  < scripts/sql/migrations/001_core_tables.sql

docker exec -i alzen_postgres psql -U alzen_admin -d alzen_db \
  < scripts/sql/migrations/002_triggers.sql

# Verify tables exist
docker exec alzen_postgres psql -U alzen_admin -d alzen_db \
  -c "\dt"
```

14.5 — Create DB connection helper for Hermes

Create `scripts/db.py`:

```python
import os
import psycopg2
from contextlib import contextmanager

@contextmanager
def get_conn():
    conn = psycopg2.connect(
        dbname=os.getenv("ALZEN_DB_NAME", "alzen_db"),
        user=os.getenv("ALZEN_DB_USER", "alzen_admin"),
        password=os.getenv("ALZEN_DB_PASSWORD"),
        host=os.getenv("ALZEN_DB_HOST", "localhost"),
        port=os.getenv("ALZEN_DB_PORT", 5432),
    )
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

14.6 — Commit

```bash
```

```
git add scripts/sql/ scripts/db.py
git commit -m "feat: phase-14 — postgresql schema migrated (leads, enrichments, emai
```

Deliverables

- 5 production tables: `leads`, `enrichments`, `emails`, `agent_events`, `patterns`
- Auto-update trigger on `leads.updated_at`
- `agent_cost_summary` view for budget monitoring
- `db.py` connection helper for Hermes scripts

PHASE 15 — Hermes Skills: Sales Department

Goal

Build and deploy three core sales-facing Hermes skills: Lead Qualifier (scores leads via DeepSeek), Enrichment Analyst (deep research), and Email Writer (A/B drafts via Qwen).

Step-by-Step

15.1 — Skill: Lead Qualifier

Create `D:/ALZEN/skills/lead-qualifier.md`:

```markdown
```

```markdown
# Skill: Lead Qualifier

## Description
Scores incoming leads 1-10 against the ICP signals in vault/patterns/icp-signals.md.
Blocks low-quality leads to protect token budget.

## Model
deepseek/deepseek-r1 (reasoning mode)

## Input
- Lead file at vault/leads/{lead_id}.md OR direct lead data dict

## Process
1. /graphify query "lead qualification icp signals"
2. Read vault/patterns/icp-signals.md
3. Evaluate lead against each signal category (Strong, Soft, Disqualification)
4. Compute score 1-10:
   - 8-10: Strongly qualified → proceed to enrichment
   - 5-7: Soft match → proceed with lower priority
   - 1-4: Disqualified → write reason, skip enrichment
5. Write to vault/leads/{lead_id}.md (update status + score)
6. Write to agent_events table (model, cost, outcome)
7. If qualified: create Paperclip task "Enrich: {lead_id}"
8. If disqualified: update leads.status = 'disqualified' in PostgreSQL

## Output
- Updated vault lead file
- PostgreSQL: leads.qual_score, leads.status updated
- Paperclip: new enrichment task (if qualified)

## Cost Control
Max 500 tokens per lead. Abort if reasoning exceeds 1000 tokens.
```

15.2 — Skill: Enrichment Analyst

Create `D:/ALZEN/skills/enrichment-analyst.md`:

```markdown
```

```
# Skill: Enrichment Analyst

## Description
Deep-dives a qualified lead to extract top 3 pain points, buying signals,
and decision-maker analysis. Uses Graphify to find past similar leads.

## Model
deepseek/deepseek-r1

## Input
- Lead ID from Paperclip task context
- vault/leads/{lead_id}.md (existing lead data)

## Process
1. /graphify query "{company_name} {industry} pain points"
2. Read vault/leads/{lead_id}.md
3. Search vault/enrichments/ for similar company profiles
4. Research via web (if Hermes web search enabled) OR reason from training
5. Extract:
   - Top 3 pain points (specific, not generic)
   - Buying signals (evidence of readiness)
   - Decision maker profile (title, likely objections)
   - Recommended email angle
6. Write enrichment to vault/enrichments/{lead_id}-enrich.md
7. Write to enrichments table (PostgreSQL)
8. Update leads.status = 'enriched'
9. Create Paperclip task "Write Email: {lead_id}"

## Cost Control
Max $0.08 per lead enrichment. Abort and mark blocker if exceeded.

## Anti-Hallucination Rules
- Only state facts that can be inferred from lead data or verified sources
- Mark uncertain claims with [inferred] tag
- Never fabricate revenue figures or employee counts
```

15.3 — Skill: Email Writer

Create `D:/ALZEN/skills/email-writer.md`:

```markdown
```

```
# Skill: Email Writer

## Description
Writes 2 A/B variant cold emails for a qualified, enriched lead.
Uses Qwen for copy quality. Reads past successful emails from vault/emails/.

## Model
qwen/qwen2.5-72b-instruct (copy quality mode)

## Input
- Lead ID from Paperclip task
- vault/enrichments/{lead_id}-enrich.md

## Process
1. /graphify query "email patterns high reply rate"
2. Read vault/enrichments/{lead_id}-enrich.md
3. Scan vault/emails/ for last 10 emails with reply_sentiment=positive
4. Identify what subject lines, angles, lengths worked best
5. Write Variant A: lead with top pain point, shorter email (<120 words)
6. Write Variant B: lead with buying signal, slightly longer (<180 words)
7. Both must:
   - Have personalized first sentence (mention specific company detail)
   - Clear single CTA (not "let me know if interested")
   - No buzzwords: synergy, leverage, paradigm, game-changer
8. Write both to vault/emails/{lead_id}-A.md and -B.md
9. Write to emails table (PostgreSQL), status = 'draft'
10. Create Paperclip task "REVIEW: Email for {company_name}" → requires Board approv

## A/B Rules
Variants must be meaningfully different in angle, not just rephrased.
Track which angle (pain point vs buying signal) performs better over time.

## Approval Gate
Emails cannot be sent without Paperclip governance gate approval.
```

15.4 — Install skills into Hermes

```bash
```

```
# Hermes auto-discovers skills in the configured skills directory
hermes skills list
# Expected: lead-qualifier, enrichment-analyst, email-writer appear

# Test qualifier skill
hermes
# Type: /lead-qualifier
# Input: test lead data
```

15.5 — Commit

```bash
git add skills/lead-qualifier.md skills/enrichment-analyst.md skills/email-writer.md
git commit -m "feat: phase-15 — hermes sales skills (qualifier, enrichment, email wr
```

Deliverables

- `lead-qualifier.md` — scores leads 1-10 with DeepSeek
- `enrichment-analyst.md` — deep research, 3 pain points
- `email-writer.md` — 2 A/B variants with Qwen
- All skills installed and visible in `hermes skills list`

PHASE 16 — Hermes Skills: Operations Department

Goal

Build two operational skills: Call Handler (categorizes reply webhooks from Instantly) and Memory Manager (weekly vault pattern analysis that improves all other skills).

Step-by-Step

16.1 — Skill: Call Handler (Reply Categorizer)

Create `D:/ALZEN/skills/call-handler.md`:

```markdown
```

```
# Skill: Call Handler

## Description
Receives webhook payloads from email send tools (e.g., Instantly.ai)
when a lead replies. Categorizes reply and routes appropriately.

## Trigger
Paperclip webhook event: email reply received

## Input
- Lead ID
- Reply text
- Original email variant (A or B)

## Sentiment Categories
- INTERESTED: Ready to talk, asked for meeting, positive language
- OBJECTION: Pushback, concern, question (needs human review)
- NOT_NOW: "Reach out in 3 months", "budget frozen", "not the right time"
- NO_FIT: Hard no, wrong person, wrong company
- AUTOMATED: OOO, bounce, automated response

## Process
1. /graphify query "reply sentiment classification"
2. Read reply text
3. Classify into one of 5 categories above
4. Update vault/calls/{lead_id}-reply.md with classification
5. Update emails.reply_sentiment in PostgreSQL
6. Update leads.status:
   - INTERESTED → 'replied' + escalate to Telegram immediately
   - OBJECTION → 'replied' + escalate to Telegram + create blocker ticket
   - NOT_NOW → 'replied' + create future follow-up routine
   - NO_FIT → 'closed'
   - AUTOMATED → ignore, keep status unchanged
7. Track winning email variant (which variant got the reply)

## Escalation (Telegram)
On INTERESTED or OBJECTION: immediately send Telegram message to operator:
"🔥 INTERESTED: {company} replied to Variant {X}. Reply: {first 100 chars}"
"⚠️ OBJECTION: {company} replied. Review needed."
```

16.2 — Skill: Memory Manager

Create `D:/ALZEN/skills/memory-manager.md`:

```markdown
# Skill: Memory Manager

## Description
Runs weekly. Analyzes the entire vault to find patterns that improve
qualification accuracy, email performance, and enrichment quality.
Updates icp-signals.md and generates a weekly learning report.

## Trigger
Paperclip scheduled routine: every Sunday 11pm

## Model
deepseek/deepseek-r1 (analysis mode)

## Process
1. /graphify scan vault/           (rebuild knowledge graph of vault)
2. /graphify god-nodes --top 20    (find most-referenced concepts)
3. Query PostgreSQL:
   - SELECT * FROM emails WHERE replied=true AND reply_sentiment='positive'
   - SELECT * FROM leads WHERE status='disqualified' GROUP BY disqual_reason
   - SELECT * FROM agent_events WHERE outcome='failed'
4. Analyze: What qualified leads converted? What email angles worked?
5. Update vault/patterns/icp-signals.md with new learnings
6. Generate vault/patterns/weekly-{date}.md with:
   - Top 3 patterns discovered
   - ICP signal updates
   - Email angle performance (A vs B win rate)
   - Recommended skill adjustments
7. Write to patterns table (PostgreSQL)
8. Rebuild Graphify graph: graphify ./vault --output-dir ./graphify
9. Notify operator via Telegram: "📊 Weekly analysis complete — 3 patterns found"

## Self-Improvement Loop
After running, flag any skill that underperformed for Paperclip review.
The operator can approve skill updates in the governance gate.
```

16.3 — Install and test skills

```bash
```

```
hermes skills list
# Expected: call-handler, memory-manager visible

# Test memory manager (dry run)
hermes run --task "Run weekly pattern analysis (dry-run)" --dry-run
```

16.4 — Commit

```bash
git add skills/call-handler.md skills/memory-manager.md
git commit -m "feat: phase-16 — hermes ops skills (call handler, memory manager)"
```

Deliverables

- `call-handler.md` — 5-category reply classification with Telegram escalation
- `memory-manager.md` — weekly learning loop that updates ICP signals
- Self-improving cycle: Reply data → Pattern analysis → ICP update → Better qualification

---

PHASE 17 — Paperclip Routines & Scheduled Heartbeats

Goal

Configure Paperclip's built-in cron scheduler to automate all recurring agency operations: lead intake processing, heartbeats, the weekly memory cycle, and the daily Graphify rebuild.

Step-by-Step

17.1 — Routine: Lead Intake Processing (every 15 min)

Paperclip → Routines → New Routine:

```
Name:        Lead Intake Processor
Trigger:     Cron — */15 * * * *  (every 15 minutes)
Assigned To: Lead Qualifier agent (Hermes)
Task Title:  "Process new leads in vault/leads/ with status=new"
Context:     {vault.leads.new_count} new leads awaiting qualification
Concurrency: 1 (no overlap)
Catch-up:    Skip (don't backfill missed runs)
```

17.2 — Routine: Email Send Queue (every 30 min)

```
Name:        Email Send Queue
Trigger:     Cron — */30 * * * *
Assigned To: Email Writer agent (Hermes)
Task Title:  "Check approved emails in PostgreSQL and send pending"
Gate:        Requires email approval (from Phase 8 Gate 1)
Concurrency: 1
```

17.3 — Routine: Reply Handler (every 5 min)

```
Name:        Reply Handler
Trigger:     Cron — */5 * * * *
             OR Webhook: POST /api/webhooks/email-reply
Assigned To: Call Handler agent (Hermes)
Task Title:  "Process incoming email replies from webhook queue"
Priority:    High (replies need fast response)
```

17.4 — Routine: Weekly Memory Analysis (Sunday 11pm)

```
Name:        Weekly Memory Analysis
Trigger:     Cron — 0 23 * * 0  (Sunday 11pm)
Assigned To: Memory Manager agent (Hermes)
Task Title:  "Run weekly vault pattern analysis, update icp-signals.md"
Timeout:     90 minutes
Notify:      Telegram on completion
```

17.5 — Routine: Daily Graphify Rebuild (3am)

```
Name:        Graphify Daily Rebuild
Trigger:     Cron — 0 3 * * *   (3am daily)
Assigned To: Graphify Scanner agent (Hermes)
Task Title:  "Rebuild Graphify knowledge graph from full workspace"
```

```
Command:     graphify ./ --output-dir ./graphify --ignore-file .graphifyignore
Notify:      Slack (future) on completion
```

17.6 — Routine: Budget Report (Monday 9am)

```
Name:        Weekly Budget Report
Trigger:     Cron — 0 9 * * 1   (Monday 9am)
Assigned To: CEO agent (Hermes)
Task Title:  "Generate weekly budget and performance report"
Output:      vault/agents/budget-report-{week}.md + Telegram summary
```

17.7 — Export routine config

```bash
npx paperclipai export --company ALZEN --include-routines > scripts/alzen-routines.j
git add scripts/alzen-routines.json
git commit -m "feat: phase-17 — paperclip routines and scheduled heartbeats configur
```

Routine Schedule Summary

Routine	Trigger	Agent
Lead Intake	Every 15 min	Lead Qualifier
Email Queue	Every 30 min	Email Writer
Reply Handler	Every 5 min / webhook	Call Handler
Memory Analysis	Sunday 11pm	Memory Manager
Graph Rebuild	Daily 3am	Graphify Scanner
Budget Report	Monday 9am	CEO

Deliverables

- 6 production routines configured in Paperclip
- All routines tied to correct agents with budgets
- Webhook trigger for real-time reply handling
- Routine config exported to version control

PHASE 18 — Telegram Mobile Control Layer

Goal

Set up the Telegram gateway so you can approve governance gates, receive agent escalations, and monitor the agency from your phone — anywhere, anytime.

Architecture

```
Hermes Gateway Process
    │
    ├── Receives: Agent escalation → sends Telegram message
    ├── Receives: Your Telegram reply → routes to correct agent
    └── Receives: Paperclip approval request → sends inline keyboard
         "Approve ✅" | "Reject ❌" | "Request Changes 📝"
```

Step-by-Step

18.1 — Create Telegram Bot

1. Open Telegram → search @BotFather
2. `/newbot` → Name: `ALZEN Control` → Username: `alzen_control_bot`
3. Copy the bot token → add to `.env`: `TELEGRAM_BOT_TOKEN=...`
4. `/setcommands` → set these commands:

```
status - Company status summary
budget - Current token spend vs limits
leads - Leads pipeline summary
tasks - Active Paperclip tasks
approve <task_id> - Approve pending governance gate
reject <task_id> - Reject pending action
pause <agent> - Pause a specific agent
resume <agent> - Resume a paused agent
graph - Latest Graphify god nodes summary
report - Trigger weekly report now
```

18.2 — Configure Hermes Telegram Gateway

```bash
hermes gateway setup

# In setup:
# Platform: Telegram
# Bot Token: [from .env]
# Allowed Users: [your Telegram user ID]
# Home Platform: telegram
```

18.3 — Start gateway

```bash
hermes gateway start
# Or as daemon:
nohup hermes gateway start > logs/telegram-gateway.log 2>&1 &
echo $! > logs/telegram-gateway.pid
```

18.4 — Create `scripts/start-gateway.sh`

```bash
#!/bin/bash
source D:/ALZEN/scripts/load-env.sh hermes

echo "📱 Starting Telegram gateway..."
hermes gateway start > D:/ALZEN/logs/telegram-gateway.log 2>&1 &
echo $! > D:/ALZEN/logs/telegram-gateway.pid
echo "✅ Telegram gateway started — message @alzen_control_bot"
```

18.5 — Connect Paperclip approval gates to Telegram

In Paperclip → Governance → Notification Settings:

```
Approval Required:    Send Telegram to operator with inline keyboard
Budget Warning:       Send Telegram alert immediately
Budget Hard Stop:     Send Telegram URGENT alert
Agent Failed:         Send Telegram with task context
New Lead Reply:       Send Telegram (INTERESTED/OBJECTION only)
```

18.6 — Test Telegram integration

```bash
# Send test message to bot
# Type: /status
# Expected: "ALZEN Company Status — Leads: X | Active Tasks: Y | Budget Used: $Z"

# Simulate approval gate
# Create test task in Paperclip requiring approval
# Expected: Telegram inline keyboard appears with Approve/Reject buttons
```

18.7 — Configure Tailscale for remote Paperclip access (optional)

```bash
# Install Tailscale for secure remote access to localhost:3100
curl -fsSL https://tailscale.com/install.sh | sh
tailscale up

# Reconfigure Paperclip to bind on Tailscale network
npx paperclipai configure --bind tailnet
# Now access dashboard from phone: http://<tailscale-ip>:3100
```

18.8 — Commit

```bash
git add scripts/start-gateway.sh
git commit -m "feat: phase-18 — telegram mobile control layer active"
```

Deliverables

- `@alzen_control_bot` live on Telegram
- Hermes gateway running, all messages routed
- Paperclip approval gates send Telegram inline keyboards
- Budget alerts, agent failures, lead replies all trigger Telegram

---

PHASE 19 — Monitoring, Logging & Alerting

Goal

Establish production-grade observability: structured logs per service, a cost dashboard, agent performance metrics, and automated alerts before problems become disasters.

Step-by-Step

19.1 — Centralized log structure

All services write to `D:/ALZEN/logs/`:

```
logs/
├── paperclip.log          — Orchestrator logs
├── hermes-tasks.log       — Task execution traces
├── telegram-gateway.log   — Telegram I/O
├── graphify-mcp.log       — Knowledge graph queries
├── graphify-watch.log     — File watch rebuilds
├── agent-costs.log        — Per-agent cost events (append-only)
└── alzen-errors.log       — Aggregated errors across all services
```

19.2 — Create log rotation config

Create `scripts/logrotate.conf`:

```
D:/ALZEN/logs/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
    copytruncate
}
```

19.3 — Add Grafana + Prometheus (optional but recommended)

Add to `docker-compose.yml`:

```yaml
```

```yaml
  prometheus:
    image: prom/prometheus:latest
    container_name: alzen_prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./scripts/prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    container_name: alzen_grafana
    ports:
      - "3001:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: Hiking@786
    depends_on:
      - prometheus
```

19.4 — Cost monitoring query (daily check)

Create `scripts/cost-report.sh`:

```bash
```

```bash
#!/bin/bash
source D:/ALZEN/scripts/load-env.sh base

echo "💰 ALZEN Cost Report — $(date)"
echo "================================"

docker exec alzen_postgres psql -U alzen_admin -d alzen_db -c "
  SELECT
    agent_name,
    COUNT(*) as events,
    SUM(tokens_used) as total_tokens,
    ROUND(SUM(cost_usd)::numeric, 4) as total_cost_usd
  FROM agent_events
  WHERE created_at > NOW() - INTERVAL '24 hours'
  GROUP BY agent_name
  ORDER BY total_cost_usd DESC;
"

echo ""
echo "📊 This Month Total:"
docker exec alzen_postgres psql -U alzen_admin -d alzen_db -c "
  SELECT ROUND(SUM(cost_usd)::numeric, 4) as month_cost_usd
  FROM agent_events
  WHERE DATE_TRUNC('month', created_at) = DATE_TRUNC('month', NOW());
"
```

19.5 — Create health check script

Create `scripts/health-check.sh`:

```bash
```

```bash
#!/bin/bash
ERRORS=0

check_service() {
  local name=$1
  local cmd=$2
  if eval "$cmd" > /dev/null 2>&1; then
    echo "✅ $name"
  else
    echo "❌ $name — DOWN"
    ERRORS=$((ERRORS+1))
  fi
}

echo "🩺 ALZEN Health Check — $(date)"
echo "================================"
check_service "PostgreSQL" "docker exec alzen_postgres pg_isready -U alzen_admin"
check_service "Redis" "docker exec alzen_redis redis-cli ping"
check_service "Paperclip" "curl -sf http://localhost:3100/health"
check_service "Graphify MCP" "curl -sf http://localhost:8889/health"
check_service "Hermes" "hermes doctor --quiet"

echo ""
if [ $ERRORS -gt 0 ]; then
  echo "⚠️  $ERRORS service(s) unhealthy"
  # Send Telegram alert
  curl -s -X POST "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/sendMessage" \
    -d "chat_id=$TELEGRAM_CHAT_ID" \
    -d "text=⚠️ ALZEN Health Check FAILED: $ERRORS service(s) down"
else
  echo "✅ All services healthy"
fi
```

19.6 — Schedule health check

```bash
# Add to crontab
crontab -e
# Add: */10 * * * * bash D:/ALZEN/scripts/health-check.sh >> D:/ALZEN/logs/health.lo
```

19.7 — Lead pipeline dashboard query

Create `scripts/pipeline-report.sh`:

```bash
#!/bin/bash
source D:/ALZEN/scripts/load-env.sh base

docker exec alzen_postgres psql -U alzen_admin -d alzen_db -c "
  SELECT status, COUNT(*) as count
  FROM leads
  GROUP BY status
  ORDER BY
    CASE status
      WHEN 'new' THEN 1
      WHEN 'qualifying' THEN 2
      WHEN 'qualified' THEN 3
      WHEN 'enriched' THEN 4
      WHEN 'emailed' THEN 5
      WHEN 'replied' THEN 6
      WHEN 'closed' THEN 7
      WHEN 'disqualified' THEN 8
    END;
"
```

19.8 — Commit

```bash
git add scripts/health-check.sh scripts/cost-report.sh scripts/pipeline-report.sh \
        scripts/logrotate.conf
git commit -m "feat: phase-19 — monitoring, health checks, cost dashboard, logging"
```

Deliverables

- Structured logs for all 5 services
- `health-check.sh` with Telegram alerting (runs every 10 min via cron)
- `cost-report.sh` for daily agent spend visibility
- `pipeline-report.sh` for leads funnel status
- Grafana + Prometheus containers (optional visual dashboard)

PHASE 20 — Security Hardening & Disaster Recovery

Goal

Harden the entire stack against credential leaks, unauthorized access, and data loss. Implement automated database backups, secret rotation procedures, and rollback capability.

Step-by-Step

20.1 — Secret audit

```bash
# Scan for any accidentally committed secrets
git log --all --oneline | head -20
git secrets --scan  # if git-secrets installed

# Scan for .env files in git history
git log --all --full-history -- "*.env"
# Should return NOTHING

# Check .gitignore is working
git check-ignore -v .env
# Expected: .gitignore:1:.env  .env
```

20.2 — File permissions hardening

```bash
# .env must never be world-readable
chmod 600 D:/ALZEN/.env
chmod 600 D:/ALZEN/.env.*

# Scripts must be executable by owner only
chmod 700 D:/ALZEN/scripts/*.sh

# Logs directory: write-only for services
chmod 755 D:/ALZEN/logs/
```

20.3 — PostgreSQL backup routine

Create `scripts/backup-db.sh`:

```bash
#!/bin/bash
source D:/ALZEN/scripts/load-env.sh base

BACKUP_DIR="D:/ALZEN/backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/alzen_db_$TIMESTAMP.sql.gz"

mkdir -p $BACKUP_DIR

echo "💾 Backing up alzen_db..."
docker exec alzen_postgres pg_dump -U alzen_admin alzen_db \
  | gzip > "$BACKUP_FILE"

echo "✅ Backup: $BACKUP_FILE"

# Keep only last 30 backups
ls -t $BACKUP_DIR/*.sql.gz | tail -n +31 | xargs rm -f 2>/dev/null

# Send confirmation to Telegram
curl -s -X POST "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/sendMessage" \
  -d "chat_id=$TELEGRAM_CHAT_ID" \
  -d "text=💾 DB Backup complete: $(basename $BACKUP_FILE)"
```

20.4 — Schedule daily backups

```bash
crontab -e
# Add:
# 0 2 * * * bash D:/ALZEN/scripts/backup-db.sh >> D:/ALZEN/logs/backup.log 2>&1
```

20.5 — Vault backup routine

Create `scripts/backup-vault.sh`:

```bash
```

```bash
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="D:/ALZEN/backups/vault_$TIMESTAMP.tar.gz"

tar -czf "$BACKUP_FILE" D:/ALZEN/vault/
echo "✅ Vault backup: $BACKUP_FILE"

# Keep last 7 vault backups
ls -t D:/ALZEN/backups/vault_*.tar.gz | tail -n +8 | xargs rm -f 2>/dev/null
```

20.6 — Paperclip rollback procedure

Paperclip supports config versioning out of the box:

```bash
# If an agent config change breaks things:
npx paperclipai rollback --agent "Lead Qualifier" --version previous
# or:
npx paperclipai rollback --company ALZEN --checkpoint last-stable
```

20.7 — Docker container restart policy

Verify all containers have `restart: unless-stopped` in `docker-compose.yml` (already set in Phase 2). This ensures containers survive system reboots.

20.8 — API key rotation procedure

Create `scripts/rotate-secrets.md`:

```markdown
```

```
# Secret Rotation Procedure

## When to rotate
- Every 90 days (routine)
- Immediately if suspected leak
- When a team member leaves

## Steps
1. Generate new API key at provider (Anthropic, OpenRouter, DeepSeek, Qwen)
2. Update D:/ALZEN/.env with new key
3. Update Hermes: hermes config set api_key <new_key>
4. Update Paperclip: npx paperclipai configure → API Keys
5. Restart all services: bash scripts/restart-all.sh
6. Revoke old key at provider
7. Commit: git commit -m "ops: rotate api keys (no values committed)"
```

20.9 — Create master startup script

Create `scripts/start-all.sh`:

```bash
```

```bash
#!/bin/bash
echo "🚀 Starting ALZEN V2.0..."

source D:/ALZEN/scripts/load-env.sh base

# 1. Docker services
echo "🐳 Starting Docker services..."
docker compose up -d postgres pgadmin redis

# 2. Wait for postgres
echo "⏳ Waiting for PostgreSQL..."
until docker exec alzen_postgres pg_isready -U alzen_admin; do sleep 2; done

# 3. Graphify
bash D:/ALZEN/scripts/start-graphify.sh

# 4. Paperclip
bash D:/ALZEN/scripts/start-paperclip.sh

# 5. Telegram Gateway
bash D:/ALZEN/scripts/start-gateway.sh

# 6. Health check
sleep 5
bash D:/ALZEN/scripts/health-check.sh

echo "✅ ALZEN V2.0 fully started"
```

20.10 — Commit

```bash
git add scripts/backup-db.sh scripts/backup-vault.sh scripts/rotate-secrets.md \
        scripts/start-all.sh
git commit -m "feat: phase-20 — security hardening, DR backups, master startup scrip
```

Deliverables

- `.env` permissions locked to `600`
- Daily automated DB backups (30-day retention)
- Daily vault backups (7-day retention)

- Secret rotation procedure documented
- `start-all.sh` master startup script
- Paperclip rollback capability verified

PHASE 21 — Production Go-Live & Full System Validation

Goal

Execute a complete end-to-end smoke test of the entire ALZEN V2.0 system. Validate every integration point, confirm the autonomous loop works, and formally declare the agency live.

Go-Live Checklist

21.1 — Pre-flight infrastructure check

```bash
bash D:/ALZEN/scripts/health-check.sh
# Expected: ✅ ALL SERVICES GREEN
```

21.2 — Inject first real lead

```bash
# Create lead file from template
cp vault/_templates/lead.md vault/leads/test-lead-001.md

# Fill with real or test lead data:
# company: TestCorp Inc
# contact: Jane Smith
# email: jane@testcorp.com
# title: VP of Sales
# company_size: 150
# funding_stage: Series B
# industry: B2B SaaS
```

21.3 — Trigger qualification manually

```bash
```

```
hermes run --task "Qualify lead: vault/leads/test-lead-001.md" --verbose
```

Expected trace:

```
[GRAPHIFY] Querying: "lead qualification icp signals"
[GRAPHIFY] Returned: {icp-signals.md, LeadQualifier, SalesDirector}
[VAULT] Reading: vault/patterns/icp-signals.md
[DEEPSEEK] Scoring lead...
[RESULT] Score: 8/10 — Series B, 150 employees, B2B SaaS ✅ QUALIFIED
[VAULT] Writing: vault/leads/test-lead-001.md (status → qualified)
[POSTGRES] UPDATE leads SET qual_score=8, status='qualified'
[PAPERCLIP] Created task: "Enrich: test-lead-001"
```

21.4 — Trigger enrichment

```bash
# Paperclip heartbeat should auto-trigger, or manually:
hermes run --task "Enrich lead: test-lead-001" --verbose
```

Expected trace:

```
[GRAPHIFY] Querying: "TestCorp SaaS pain points similar companies"
[DEEPSEEK] Analyzing...
[RESULT] Pain points: [scaling SDR team, lead routing chaos, CRM hygiene]
[VAULT] Writing: vault/enrichments/test-lead-001-enrich.md
[POSTGRES] INSERT INTO enrichments (...)
[PAPERCLIP] Created task: "Write Email: test-lead-001"
```

21.5 — Trigger email generation

```bash
hermes run --task "Write email for test-lead-001" --verbose
```

Expected trace:

```
[GRAPHIFY] Querying: "email patterns high reply rate"
[VAULT] Reading: vault/enrichments/test-lead-001-enrich.md
[QWEN] Writing Variant A (pain point: scaling SDR team)...
[QWEN] Writing Variant B (angle: buying signal)...
[VAULT] Writing: vault/emails/test-lead-001-A.md
[VAULT] Writing: vault/emails/test-lead-001-B.md
[POSTGRES] INSERT INTO emails (variant A, variant B)
[PAPERCLIP] Created approval gate: "REVIEW: Email for TestCorp"
[TELEGRAM] 📧 Email drafts ready for TestCorp. Approve? ✅ ❌
```

21.6 — Approve email via Telegram

On your phone:

- Receive Telegram message with approval keyboard
- Tap ✅ Approve
- System marks email as approved, queues for send

21.7 — Simulate reply webhook

```bash
curl -X POST http://localhost:3100/api/webhooks/email-reply \
  -H "Content-Type: application/json" \
  -d '{
    "lead_id": "test-lead-001",
    "reply_text": "Hi, yes this is relevant! Can we jump on a call this week?",
    "email_variant": "A"
  }'
```

Expected:

```
[HERMES Call Handler] Classifying reply...
[DEEPSEEK] Sentiment: INTERESTED
[VAULT] Writing: vault/calls/test-lead-001-reply.md
[POSTGRES] UPDATE emails SET replied=true, reply_sentiment='positive'
[POSTGRES] UPDATE leads SET status='replied'
[TELEGRAM] 🔥 INTERESTED: TestCorp replied to Variant A. "Hi, yes this is
relevant!..."
```

21.8 — Validate Graphify reflects new state

```bash
# Trigger incremental graph update
graphify ./vault --output-dir ./graphify

# Query graph for the lead
curl http://localhost:8889/mcp/query?q=test-lead-001
# Expected: Node appears with edges to enrichment, emails, calls
```

21.9 — Validate cost tracking

```bash
bash D:/ALZEN/scripts/cost-report.sh
# Expected: Shows costs for Lead Qualifier, Enrichment Analyst, Email Writer
# All costs within budget limits
```

21.10 — Validate audit trail

```bash
docker exec alzen_postgres psql -U alzen_admin -d alzen_db -c "
  SELECT agent_name, event_type, outcome, cost_usd
  FROM agent_events
  ORDER BY created_at DESC
  LIMIT 10;
"
# Expected: 4-5 events for the test lead (qualify, enrich, email, handle-reply)
```

21.11 — Final Git tag

```bash
git add .
git commit -m "feat: phase-21 — production validation complete, ALZEN V2.0 live"
git tag -a v2.0.0-live -m "ALZEN V2.0 production go-live — all 21 phases complete"
git push origin main --tags
```

21.12 — Send go-live announcement to Telegram

```bash
```

```
curl -s -X POST "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/sendMessage" \
  -d "chat_id=$TELEGRAM_CHAT_ID" \
  -d "text=🚀 ALZEN V2.0 IS LIVE

✅ Graphify: Knowledge graph active
✅ Paperclip: Orchestrator running
✅ Hermes: 7 agents deployed
✅ PostgreSQL: Schema live
✅ Vault: Memory operational
✅ Telegram: Mobile control active
✅ Routines: All schedules running
✅ Governance: Approval gates enforced

First lead qualified, enriched, emailed, and replied in test run.

The agency is open for business."
```

ALZEN V2.0 — Full System Map (Final State)

```
┌──────────────────────────────────────────────────────────────────────┐
│                    ALZEN V2.0 PRODUCTION SYSTEM                      │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │         LAYER 1: GRAPHIFY KNOWLEDGE GRAPH (Port 8889)        │    │
│  │    graph.json ← live-watch ← all repos + vault + scripts     │    │
│  │    MCP server → Hermes PreToolUse hook → 71.5× token savings  |    |
│  └──────────────────────────────────────────────────────────────┘    │
│                              ▲ queries                               │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │         LAYER 2: PAPERCLIP ORCHESTRATOR (Port 3100)          │    │
│  │    Org Chart: CEO → Sales Dir → 5 specialist agents          │    │
│  │    Budgets: $500/mo hard cap, per-agent limits               │    │
│  │    Governance: 5 approval gates, full audit log              │    │
│  │    Routines: 6 cron schedules, webhook triggers              │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                           ▲ heartbeats                               │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │         LAYER 3: HERMES AGENT WORKFORCE (Python)             │    │
│  │    Main Agent: DeepSeek (reasoning) + Qwen (email)           │    │
│  │    Subagents: up to 10 parallel workers per batch            │    │
│  │    Skills: qualifier, enrichment, email, call, memory        │    │
```

```
│  │    Memory: persistent vault + cross-session recall           │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                    ▲ reads/writes                                    │
│  ┌──────────────────────────────┐  ┌──────────────────────────────┐   │
│  │  LAYER 4A: OBSIDIAN VAULT │  │  LAYER 4B: POSTGRESQL 5432  │   │
│  │  vault/leads/ — intakes   │  │  leads — qualified state     │   │
│  │  vault/enrichments/       │  │  enrichments — pain points   │   │
│  │  vault/emails/            │  │  emails — A/B variants       │   │
│  │  vault/calls/             │  │  agent_events — cost audit   │   │
│  │  vault/patterns/ — ICP    │  │  patterns — weekly learning  │   │
│  └──────────────────────────┘  └──────────────────────────────┘   │
│                                                                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │         LAYER 5: MOBILE CONTROL (Telegram)                   │   │
│  │    Approval gates → inline keyboard on phone                 │   │
│  │    Budget alerts → instant Telegram push                     │   │
│  │    Agent escalations → INTERESTED/OBJECTION replies          │   │
│  └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘


AUTONOMOUS LEAD PIPELINE:
New Lead → [15min] Qualify → [heartbeat] Enrich → [heartbeat] Email Draft
→ [Telegram] Human Approve → Send → [webhook] Reply → Classify
→ INTERESTED: Telegram alert → Human closes deal
→ Sunday: Memory Manager learns → updates ICP → better qualification next week
```

QUICK REFERENCE: All Service Ports

Service	Port	URL
Paperclip Dashboard	3100	http://localhost:3100
Graphify MCP	8889	http://localhost:8889
PostgreSQL (ALZEN)	5432	psql / pgAdmin
pgAdmin	5050	http://localhost:5050
Redis	6379	redis-cli
Grafana (optional)	3001	http://localhost:3001
Prometheus (optional)	9090	http://localhost:9090

Service	Port	URL
Paperclip embedded DB	54329	Internal only

QUICK REFERENCE: Key File Paths

File	Purpose
`D:/ALZEN/.env`	All secrets (gitignored)
`D:/ALZEN/scripts/start-all.sh`	Boot entire stack
`D:/ALZEN/scripts/health-check.sh`	System health + Telegram alert
`D:/ALZEN/graphify/GRAPH_REPORT.md`	Knowledge graph summary
`D:/ALZEN/vault/patterns/icp-signals.md`	ICP qualification rules
`D:/ALZEN/skills/*.md`	All Hermes agent skills
`~/.hermes/SOUL.md`	Hermes agent persona
`D:/ALZEN/docker-compose.yml`	Infrastructure definition

QUICK REFERENCE: Daily Operations

```bash
```

```bash
# Start everything
bash D:/ALZEN/scripts/start-all.sh

# Health check
bash D:/ALZEN/scripts/health-check.sh

# Cost report
bash D:/ALZEN/scripts/cost-report.sh

# Pipeline status
bash D:/ALZEN/scripts/pipeline-report.sh

# Rebuild knowledge graph
graphify ./ --output-dir ./graphify

# Backup DB
bash D:/ALZEN/scripts/backup-db.sh

# Restart a specific agent in Paperclip
npx paperclipai agent restart "Lead Qualifier"
```

ALZEN V2.0 — Built on Paperclip + Hermes + Graphify. 21 phases. Zero human bottlenecks.